Incremental Learning for LLM-based Tokenization and Recommendation

Haihan Shi
University of Science and Technology of China
Hefei, China
shh924@mail.ustc.edu.cn

Xinyu Lin
National University of Singapore
Singapore
xylin1028@gmail.com

Wenjie Wang
University of Science and Technology of China
Hefei, China
wenjiewang96@gmail.com

Wentao Shi
University of Science and Technology of China
Hefei, China
shiwentao123@mail.ustc.edu.cn

Junwei Pan
Independent Researcher
Shenzhen, China
pandevirus@gmail.com

Jiang Jie
Independent Researcher
Shenzhen, China
13965030@qq.com

Fuli Feng*
University of Science and Technology of China
Hefei, China
fulifeng93@gmail.com

Abstract

Large Language Models for Recommendation (LLM4Rec) have shown great potential. Many LLM4Rec approaches technically leverage a learnable tokenizer to assign item identifiers and then enable a Recommender LLM (RecLLM) to process tokenized items and user interactions for recommendation. However, a key challenge in their real-world deployment is the need for continuous retraining over time to accommodate new items and evolving user interests. While existing retraining methods can be applied to RecLLMs, learnable tokenizers introduce additional retraining challenges. We conduct a comprehensive investigation into the joint retraining of RecLLMs and learnable tokenizers, identifying key issues such as identifier collision and identifier shifts across periods. To address these, we propose Reformer, an incremental learning framework to fine-tune RecLLMs and learnable tokenizers at each period. Reformer employs a dynamic codebook to mitigate identifier collision by appending new codes and enforcing a diversity-oriented code assignment constraint. Additionally, Reformer adopts an identifier freezing strategy to ensure the invariance of previously assigned item identifiers across retraining periods. We instantiate Reformer on two representative RecLLMs and conduct extensive experiments on three real-world datasets. Substantial results demonstrate its superior retraining performance, facilitating the real-world deployment of LLM4Rec.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

LLM-based Recommendation; Incremental Learning; Tokenizer Retraining

ACM Reference Format:
Haihan Shi, Xinyu Lin, Wenjie Wang, Wentao Shi, Junwei Pan, Jiang Jie, and Fuli Feng. 2025. Incremental Learning for LLM-based Tokenization and Recommendation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3746252.3761385

1 Introduction

Large Language Models for Recommendation (LLM4Rec) have been receiving increasing attention in both academia and industry [15, 16, 25]. Existing efforts typically utilize a tokenizer to convert users' interactions into token sequences [2, 8, 20, 30], and then input the token sequences into a Recommender LLM (RecLLM) for user behavior reasoning and next-item generation [7]. Despite their success, previous work has rarely considered the task of retraining LLM4Rec for real-world deployment. Given the continuous emergence of numerous new items and interactions each day, it is essential to retrain LLM4Rec with fresh data. This enables the RecLLM to understand new items and capture users' evolving interests for accurate recommendations [1, 24].

Retraining LLM4Rec involves updating both the tokenizer and the RecLLM. Specifically, tokenizers can be categorized as either unlearnable or learnable. For unlearnable tokenizers (*e.g.,* those using item titles as identifiers [2]), LLM4Rec retraining only

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '25, Seoul, Republic of Korea.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3761385

requires updating the RecLLM periodically, which can be achieved through fine-tuning [23, 29] or full retraining [12, 37]. Fine-tuning updates the model using only new interaction data from the current period, whereas full retraining incorporates the whole historical data and new data for training [12, 23, 29, 37]. In contrast, retraining RecLLMs with learnable tokenizers (*e.g.,* codebook-based tokenizers [20, 30, 39]) is more complex, as it requires maintaining alignment between the RecLLM and the tokenizer.

To elaborate, we comprehensively explore the potential of applying existing retraining methods on RecLLMs and learnable tokenizers, and present the key findings as follows:

- It is essential to retrain both RecLLMs and tokenizers. Retraining only the tokenizer prevents RecLLMs from properly interpreting new item identifiers, making it difficult to understand user behaviors and recommend new items (see evidence in Section 3.2). Conversely, if only the RecLLM is retrained, the tokenizer may fail to generalize to new items. Moreover, due to identifier assignment bias [30] and the continuous emergence of new items, freezing the tokenizer will lead to severe identifier collision, *i.e.,* different items share the same identifier (see theoretical and empirical evidence in Sections 3.2).
- Jointly retraining the tokenizer and RecLLM also presents challenges. After fine-tuning or fully retraining the tokenizer, item identifiers from previous periods may change, disrupting RecLLM's original understanding and generation (see analysis in Section 3.3). This necessitates full retraining of RecLLM on all historical interactions in previous periods to realign past items with their new identifiers. However, due to the large number of RecLLM parameters and the vast historical interactions, this process incurs a prohibitively high computational cost [18].

Building on these insights, we deduce that retraining RecLLMs with learnable tokenizers should pursue the following objectives: 1) both the tokenizer and RecLLM must be periodically retrained to adapt to new items while preventing identifier collision; and 2) in each period, the RecLLM and tokenizer should be fine-tuned rather than fully retrained for efficiency. However, it is crucial to ensure that the identifiers of previous items remain unchanged across periods (*i.e.,* identifier invariance), thereby avoiding the need for the RecLLM to realign past items with their new identifiers.

To this end, we propose a novel inc**R**emental l**e**arning **f**ramew**o**rk for **R**ecLL**M**s with the learnable tokeniz**er**, named Reformer. Given the superior scalability and design rationality of codebook-based tokenizers over text-based and ID-based learnable tokenizers [14, 27] (see Section 3.1), we primarily devise Reformer for RecLLMs with codebook-based tokenizers.[1] Specifically, Reformer fine-tunes RecLLMs and tokenizers with new interaction data at each period. To prevent identifier collision, Reformer introduces a dynamic codebook with a frequency-based diversity constraint. The dynamic codebook expands the identifier space by appending new codes at each period, theoretically enhancing the tokenizer's capacity. Besides, the diversity constraint leverages code assignment frequency to promote the activation of low-frequency codes during tokenizer fine-tuning, thereby mitigating code assignment bias and generating more distinguishable identifiers. Moreover, to ensure item identifier invariance, Reformer adopts an identifier freezing strategy: once an item is tokenized, its identifier is stored in a database and remains unchanged in subsequent periods.

We instantiate Reformer on representative T5-based TIGER [20] and QWen1.5B-based LC-Rec [39], and conduct extensive comparisons with existing retraining strategies on three datasets. Substantial results validate the superiority of Reformer across different retraining periods. We release codes and data at **https://github.com/shh9249/Reformer**. The main contributions of this work are threefold:

- We focus on a critical yet under-explored task: the retraining of RecLLMs with learnable tokenizers. Through comprehensive theoretical and empirical analyses, we identify two major challenges in joint retraining: identifier collision and identifier shifts across periods for existing items.
- We propose Reformer, a novel incremental learning framework. To alleviate identifier collision, Reformer adopts a dynamic codebook to theoretically enhance the tokenizer's capacity and a frequency-based diversity constraint to mitigate code assignment bias. Besides, Reformer utilizes an identifier freezing strategy to ensure identifier invariance across periods.
- We instantiate Reformer on two representative RecLLMs with learnable tokenizers and perform extensive experiments on three real-world datasets, demonstrating the superior retraining effectiveness compared to various baselines.

2 Task Formulation

• **LLM-based Generative Recommendation**. LLM4Rec typically involves two key steps, *i.e.,* item tokenization and next-item generation. Precisely, item tokenization employs a tokenizer to assign each item $i \in \mathcal{I}$ a unique identifier $\tilde{i}$. To recommend the next item, given a user's historical interactions $\mathcal{S}_u = [i_1, i_2, \ldots, i_L]$, where $u \in \mathcal{U}$ and $L = |\mathcal{S}_u|$, it is transformed into the identifier sequence $\tilde{\mathcal{S}}_u = [\tilde{i}_1, \tilde{i}_2, \ldots, \tilde{i}_L]$. Based on the transformed sequence, the RecLLM $M_\theta(\cdot)$ autoregressively generates the next item identifier as the recommended item.

• **Incremental Learning of LLM4Rec**. In real-world recommendation scenarios, new users, items, and interactions continuously stream. Formally, we denote the data stream as $\{\mathcal{D}_0, \mathcal{D}_1, \ldots, \mathcal{D}_p, \ldots\}$, where p denotes the p-th time period. We denote the newly introduced items for each time period as $\{\mathcal{I}_0, \mathcal{I}_1, \ldots, \mathcal{I}_p, \ldots\}$. According to the data used for retraining $\mathcal{D}_r$ in period p, incremental learning mainly encompasses two widely employed strategies to serve for the next time period $p+1$. 1) **Fine-tuning** utilizes the latest interaction data in the current time period $\mathcal{D}_r = \mathcal{D}_p$ to update the model. 2) **Full retraining** utilizes all previous data $\mathcal{D}_r = \mathcal{D}_{\leq p}$ for the update. In particular, for unlearnable tokenizers (*e.g.,* item titles), existing retraining strategies can be applied directly to RecLLMs. However, incremental learning with learnable tokenizers is challenging due to the joint retraining of the tokenizer and RecLLM. To address this, we analyze exsiting strategies in Section 3 and propose a dedicated incremental learning framework for learnable tokenizers in Section 4.

[1] As an initial attempt at retraining RecLLMs with learnable tokenizers, we leave the adaptation of Reformer to text-based and ID-based learnable tokenizers, such as IDGenRec [27], for future work.

3 Preliminary Analysis

In this section, we conduct an in-depth analysis to investigate the potential of retraining for LLM4Rec with learnable tokenizers. Specifically, we seek to explore the potential of 1) only retraining the tokenizer or the RecLLM, respectively; and 2) retraining both the tokenizer and the RecLLM.

3.1 Analysis Settings

- **Streaming Data.** To simulate the incremental learning setting, we sort the data by timestamp and split interactions into five periods with approximately equal numbers of new items per period to ensure a fair tokenizer performance comparison. Detailed descriptions of the dataset can be found in Section 5.1.
- **Learnable Tokenizer.** LLM4Rec with learnable tokenizers employs a trainable model to generate identifiers, including ID identifiers [14], textual identifiers [27] , and codebook-based identifiers [20]. Among them, codebook-based tokenizers hierarchically represent items from coarse-grained features to fine-grained details, aligning better with autoregressive generation and demonstrating strong performance. As such, we investigate codebook-based tokenizers for analysis. Specifically, we implement a representative tokenizer (*i.e.,* TIGER [20]) with RQ-VAE [11]. For each item, its semantic embedding $\boldsymbol{s}$ is extracted via a pre-trained language model and encoded into latent embedding $\boldsymbol{e} \in \mathbb{R}^d$ by a learnable encoder. $\boldsymbol{e}$ is then quantized through a J-level codebook to generate identifiers, *i.e.,* code sequences. For the j-th level codebook $C_j = \{\boldsymbol{v}_{c,j}\}_{c=1}^{N_j}$, where $\boldsymbol{v}_{c,j} \in \mathbb{R}^d$ is the trainable code vector, and N_j is the size of the codebook, the j-th quantization process can be represented as:

$$\begin{cases} c_j = \arg\min_c \|\boldsymbol{r}_{j-1} - \boldsymbol{v}_{\boldsymbol{c},\boldsymbol{j}}\|^2, \quad v_c \in C_j, \\ \boldsymbol{r}_j = \boldsymbol{r}_{j-1} - \boldsymbol{v}_{\boldsymbol{c}_j,\boldsymbol{j}}, \end{cases} \tag{1}$$

where c_j is the quantized codebook vector index, r_{j-1} is the quantization residual. After recursive quantization, the token sequence can be obtained as $\tilde{i} = [c_1, c_2, \ldots, c_J]$, along with the quantized embedding $\tilde{e} = \sum_{j=1}^{J} \boldsymbol{v}_{\boldsymbol{c_j},\boldsymbol{j}}$, which is then reconstructed by a learnable decoder into $\hat{s}$. The training loss can be expressed as:

$$\mathcal{L}_{\text{tokenizer}} = \mathcal{L}_{\text{Recon}} + \mathcal{L}_{\text{RQ-VAE}}, \tag{2}$$

where

$$\begin{cases} \mathcal{L}_{\text{Recon}} = \|\mathbf{s} - \hat{\mathbf{s}}\|^2, \\ \mathcal{L}_{\text{RQ-VAE}} = \sum_{j=1}^{J} \|\text{sg}[\mathbf{r}_{j-1}] - v_{c_j,j}\|^2 + \mu\|\mathbf{r}_{j-1} - \text{sg}[v_{c_j,j}]\|^2, \end{cases} \tag{3}$$

where $sg[\cdot]$ denotes the stop-gradient operation, and the balancing coefficient μ regulates optimization between codebook vectors and encoder parameters. The reconstruction loss $\mathcal{L}_{\text{Recon}}$ preserves latent semantic information while $\mathcal{L}_{\text{RQ-VAE}}$ minimizes the residual errors and jointly trains both the encoder and the codebook.

- **RecLLM**. Following previous works [7, 8], we adopt T5-small as RecLLM for analysis and retrain RecLLM by maximizing the log-likelihood, *i.e.,* $\log P_\theta(y_\tau | y_{<\tau}, \tilde{\mathcal{S}}_u)$, where y_τ is the τ-th code of identifier, and $y_{<\tau}$ denotes the code sequence preceding y_τ. Detailed implementations can be found in Section 5.1.

3.2 Necessity of Joint Retraining

To investigate the traditional single model retraining in LLM4Rec, we retrain 1) the tokenizer only and 2) RecLLM only, at each time period p respectively, and evaluate them on the next time period $\mathcal{D}_{p+1}$. The result is presented in Table 1.

Table 1: Performance comparison of LLM4Rec without retraining, tokenizer fine-tuning, and RecLLM fine-tuning. "Tok." and "FT" denotes tokenizer and fine-tuning, respectively. "×" means that the RecLLM or tokenizer is frozen.

Method		Period 1		Period 2		Period 3		Period 4	
LLM	Tok.	R@5	N@5	R@5	N@5	R@5	N@5	R@5	N@5
×	FT	**0.0012**	**0.0017**	0.0014	0.0012	0.0004	0.0004	0.0003	0.0001
FT	×	**0.0012**	**0.0017**	0.0062	0.0080	0.0037	0.0036	0.0098	0.0087
FT	FT	**0.0012**	**0.0017**	**0.0080**	**0.0107**	**0.0044**	**0.0042**	**0.0104**	**0.0090**

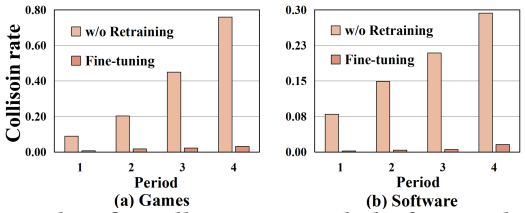


Figure 1: Identifier collision rates with the frozen tokenizer across different periods on Games and Software datasets.

- **Tokenizer Retraining with Frozen RecLLM**. From Table 1, we can find that retraining the tokenizer alone leads to poor performance, as RecLLMs struggle to understand new identifiers, thus failing to generate the newly introduced items properly (*cf.* Section 1). Thus, retraining RecLLM is crucial for accurately understanding and generating new items.
- **RecLLM Retraining with Frozen Tokenizer**. Despite the significance of RecLLM retraining, freezing the tokenizer that is initially trained in period 0 might suffer from the identifier collision issue. To elaborate, codebook-based tokenizers are prone to assign imbalanced tokens, biasing towards tokens assigned to popular items [30]. As new items continuously stream, the popular tokens will dominate the identifier assignments, potentially leading to identifier collisions, *i.e.,* multiple items are assigned with the same identifier. To validate this suspicion, we empirically and theoretically assess the collision rate of identifiers.

Specifically, we define **collision rate** to measure the conflicts of the identifiers. At each time period, we utilize a ready-to-serve tokenizer (*e.g.,* frozen or retrained tokenizer) to assign identifiers to all items. We then obtain the collision rate for the time period t via

$$col_rate = \frac{\sum_{i \in \mathcal{I}_p} \sum_{j \in \mathcal{I}_{<p}} \mathbb{I}(\tilde{i} = \tilde{j})}{|\mathcal{I}_p|}, \tag{4}$$

where $\mathcal{I}_p$ denotes newly introduced items in period t and $\mathbb{I}(\tilde{i} = \tilde{j})$ returns 1 if two identifiers collide otherwise 0.

Identifier Collision Issue. Figure 1 shows the collision rates of the frozen tokenizer ("w/o Retraining") and the retrained tokenizer ("Fine-tuning"). We can observe that the collision rate of the frozen tokenizer gradually increases, indicating a progressive decline in identifier distinguishability as new items continuously flow in. Conversely, the retrained tokenizer maintains low collision rates across all periods. Moreover, based on the frozen tokenizer, the fine-tuned RecLLM fails to yield satisfying results as shown in Table 1. The above empirical evidence confirms the necessity of retraining the tokenizer in incremental learning of LLM4Rec. We then take a further step to investigae **why the identifier collision**

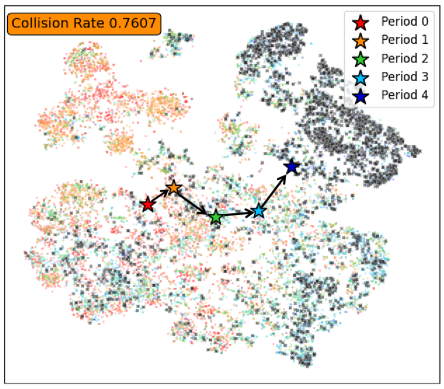


Figure 2: A t-SNE visualization of the item semantic embeddings extracted from the frozen tokenizer across all periods on Games. The frozen tokenizer is initially trained on period 0, and stars denote the center of embeddings for each period.

issue happens by visualizing the semantic embeddings of newly introduced items in different periods as shown in Figure 2, where we can observe that the item feature gradually shifts over time due to environmental factors, which is consistent with findings in [19]. As such, the frozen tokenizer fails to distinguish between items as item feature gradually shifts to the underrepresented space (*e.g.,* items with conflicted identifiers in period 4 are predominantly centered in the underrepresented top-right and bottom-right regions). This explains the exacerbated collision issues over time.

Theoretical Analysis. In addition to empirical analysis, by making several assumptions about the distribution of item semantic embedding s in different data streams I, we also conduct theoretical analysis for the collision rate.

Theorem 3.1 (Collision Rate With Frozen Tokenizer). *Let $C \subset \mathbb{S}^{d-1}$ denote a codebook uniformly distributed over the unit sphere in $\mathbb{R}^d$, optimized to achieve uniform quantization assignments for data stream $\mathcal{I}_1$, whose semantic embeddings follow a Gaussian distribution $\mathcal{N}(0, \sigma^2\mathbf{I})$. Consider a second data stream $\mathcal{I}_2$ with embeddings distributed as $\mathcal{N}(\mathbf{u}, \sigma^2\mathbf{I})$, where $\mathbf{u} \in \mathbb{R}^d$ is a fixed mean vector. If C remains unmodified, the probability of quantization conflict is lower bounded by:*

$$col_rate \geq 1 - (1 - P_{k_min})^{|\mathcal{I}_2|} - |\mathcal{I}_2| P_{k_min} (1 - P_{k_min})^{|\mathcal{I}_2|-1}, \quad (5)$$

$$P_{k_min} = 1 - \exp\left(-\frac{1}{2}\left(\frac{|\mathbf{u}|}{\sigma} - \cot\theta\sqrt{d-1}\right)^2\right). \quad (6)$$

where col_rate depends on σ, $\mathbf{u}$, and the parameter θ is determined by the size of codebook $|C|$.

The proof can be found in Appendix B. Eq (5) shows that as $||\mathbf{u}|| \to +\infty$, the collision rate converges to 1, indicating that greater distributional discrepancy between the two data streams inevitably increases the collision rate when retaining the original codebook.

Conclusion 3.1. *Based on the above insights, it is crucial to retrain both the tokenizer and the RecLLM to facilitate the assignment of distinguishable identifiers and an accurate understanding of new items for proper recommendation.*

Table 2: Performance of RecLLM on warm interactions with original and shifted identifiers.

Method	Identifier	R@5	R@10	N@5	N@10
TIGER	Original	0.0026	0.0084	0.0019	0.0041
	Shifted-FT	0.0001	0.0010	0.0002	0.0005
	Shifted-FR	0.0005	0.0020	0.0003	0.0009

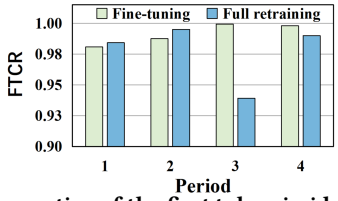


Figure 3: Change ratios of the first token in identifiers before and after tokenizer retraining across all periods on Games.

3.3 Challenges in Joint Retraining

In this section, we empirically demonstrate that the identifier distributions undergo significant changes between adjacent stages in both fine-tuning and full retraining strategies, which complicates the incremental learning of the RecLLM.

- **Joint Retraining of RecLLM and Tokenizer**. For joint retraining, the tokenizer is first retrained each period to assign new distinguishable identifiers and refresh existing identifiers, followed by RecLLM retraining on transformed interactions. However, tokenizer retraining may cause identifier shifts, altering existing items' identifiers due to parameter updates. This outdates RecLLM's knowledge of items and user behaviors, requiring full retraining to realign identifiers and relearn item and user behaviors.
- **Identifier Shifts Issue**. To empirically study the identifier shifts before and after tokenizer retraining, we employ two strategies, *i.e.,* fine-tuning and full retraining, respectively. For each strategy, we compute the change ratio of the first token to assess the inconsistency between identifiers with and without updates. The token change ratio is obtained via

$$change_ratio = \frac{\sum_{i \in \mathcal{I}_{p-1}} \mathbb{I}(\tilde{i}_1 = \tilde{i}'_1)}{|\mathcal{I}_{p-1}|}, \quad (7)$$

where $\tilde{i}_1$ and $\tilde{i}'_1$ denote the first token of identifier before and after tokenizer retraining, respectively. From the results presented in Figure 3, we can observe that both fine-tuning and full retraining strategies exhibit significant changes in identifiers (more than 93%).

To further investigate the impact of these changes on RecLLM performance, we train a tokenizer T_0 on $\mathcal{I}_0$ and update it using fine-tuning and full retraining. We then train a RecLLM on $\mathcal{D}_0$ tokenized with T_0. Based on the three tokenizers, we tokenize items I_0 and obtain original identifiers and shifted identifiers with fine-tuning ("Shifted-FT") and full retraining ("Shifted-FR"), respectively. We then evaluate RecLLM on the user interactions with $i \in I_0$ in period 1 based on original and shifted identifiers to assess whether the LLM can understand items after the identifier shifts. As shown in Table 2, RecLLM fails to comprehend the shifted identifiers, resulting in a significant performance decline.

Conclusion 3.2. *Directly applying fine-tuning or full retraining incremental learning strategies to the tokenizer leads to significant*

distribution shifts in the identifiers of existing items across adjacent periods. Consequently, the RecLLM is forced to relearn and interpret new identifiers for previously encountered items, thereby incurring higher incremental learning costs.

• **Summary**. Based on the above insights, we consider two primary objectives for tokenizer retraining for incremental learning of LLM4Rec. (1) Both the tokenizer and RecLLM must be periodically retrained to adapt to new items while alleviating the identifier collision issue. (2) The existing identifiers should remain unchanged across different periods, to facilitate fine-tuning of RecLLM, which efficiently updates the model with the most recent data.

4 Method

To accomplish the incremental learning task and achieve the primary objectives, we propose Reformer, a novel framework that combines: (1) dynamic codebook expansion, (2) frequency-based diversity constraint, and (3) historical identifier freezing strategy. This integrated approach provides a simple yet effective solution for the joint retraining of the tokenizer and RecLLM.

• **Dynamic Codebook Expansion.** To alleviate the issue of identifier conflicts, Reformer performs dynamic codebook expansion at each period. Specifically, a predefined number of new codes are appended to the codebook and initialized by the embeddings of the first training batch of new items. After initialization, both the preserved existing codes and new codes participate in joint training. The expansion process can be formulated as:

$$C_{p,j} = C_{p-1,j} \cup C^{'}_{p,j}, \tag{8}$$

where $C^{'}_{p,j}$ represents the new codes of the j-th layer codebook at period p with a expansion size N_p, *i.e.* $|C^{'}_{p,j}| = N_p$. This expansion mechanism enables scaling the codebook for the growing items, effectively preventing identifier conflicts. Notably, the parameter growth remains acceptable because of the high encoding efficiency.

• **Frequency-Based Diversity Constraint.** We propose the frequency-based diversity constraint to avoid identifier conflicts and enhance code diversity. Concretely, we introduce a penalty term based on the code assignment frequency during the codebook allocation process. Given the new item i's residual vector $\boldsymbol{r}_{i,j-1}$ at period p and the codebook vector $\boldsymbol{v}_{c,j}$ at the j-th quantization process of the tokenizer, we incorporate the frequency-based penalty term into the distance calculation:

$$dist(\boldsymbol{r}_{i,j-1}, \boldsymbol{v}_{c,j}) = (1 + \boldsymbol{v}_f[c]^{penalty(\boldsymbol{v}_{c,j})}) \cdot \|\boldsymbol{r}_{i,j-1} - \boldsymbol{v}_{c,j}\|^2 \tag{9}$$

where $v_f[c]$ can be calculated by:

$$\boldsymbol{v}_f[c] = \frac{\sum_{i \in \mathcal{I}_f} \mathbb{I}_a(i,c,j)}{|\mathcal{I}_f|}, \tag{10}$$

and

$$\mathbb{I}_a(i,c,j) = \begin{cases} 1, & if \ \ \tilde{i}[j] = c, \\ 0, & if \ \ \tilde{i}[j] \neq c, \end{cases} \tag{11}$$

where $\mathbb{I}_a(i,c,j)$ demonstrates whether the j-th token of item i's identifier is assigned to codebook index c in the j-th layer codebook, $\mathcal{I}_f$ contains items from the previous tokenizer training batch and from all historical periods. $\boldsymbol{v}_f$ is the normalized frequency vector computed on the $\mathcal{I}_f$, and $penalty(\boldsymbol{v}_{c,j})$ determines the penalty strength which is stronger when $penalty(\boldsymbol{v}_{c,j})$ is smaller. To better adapt to item migration across different periods, the value of $penalty(\boldsymbol{v}_{c,j})$ differs based on whether the codebook vector belongs to a new codebook, which can be formulated as:

$$penalty(\boldsymbol{v}_{i,j}) = \begin{cases} \alpha\,, \ \boldsymbol{v}_{i,j} \in C^{'}_{p,j} \\ \beta\,, \ \boldsymbol{v}_{i,j} \in C_{p-1,j} \end{cases} \tag{12}$$

where α and β are hyperparameters. By configuring α to exceed β, we impose a reduced penalty strength on the new codebook, thereby facilitating the migration of new items towards the appended codes.

• **Historical Identifier Freezing Strategy.** To solve the issue of identifier shifts, we implement a historical identifier freezing strategy. Once an item i is tokenized at period p, its identifier is stored in a look-up table and remains unchanged:

$$\tilde{i}_p = \tilde{i}_{p+1} = \dots \tag{13}$$

This strategy effectively avoids unnecessary relearning of established item-identifier mappings, thereby enhancing Reformer's efficiency and stability for dynamic recommendation systems.

5 Experiment

We conduct extensive experiments on three datasets to answer three questions: **RQ1:** How does Reformer perform compared to existing retraining methods on different backbones? **RQ2:** How do the components of Reformer affect performance? **RQ3:** How does Reformer perform under different settings (*e.g.,* hyperparameters)? **RQ4:** How is the Reformer's time efficiency performance?

5.1 Experimental Setting

• **Datasets**. We adopt three real-world datasets: (1) Games and (2) Software from Amazon datasets, and a video games dataset (3) Steam [9]. These datasets contain rich item semantic information (*e.g.,* titles) for codebook-based tokenizers. Each dataset is chronologically divided into 5 periods with equal new items per period to simulate incremental learning scenarios. The dataset statistics and Relative Interaction Ratio (RIR) of each period to period 0 are listed in Table 3. The results of Games dataset divided into 5 periods with equal interactions can be found in Appendix A.

Table 3: Statistics of the datasets.

Datasets	Users	Items	Avg. Length	RIR
Games	39,327	18,857	9.04	1.0: 1.6: 4.0: 3.3: 2.0
Software	64,162	10,992	8.26	1.0: 2.0: 3.3: 4.6: 3.6
Steam	32,916	14,350	43.62	1.0: 1.4: 1.2: 1.1: 1.2

• **Baselines and Backbones**. We implement two commonly used retraining strategies (1) **Fine-tuning**, (2) **Full retraining** with a control group (3) **w/o Retraining** and independently apply them on RecLLM and tokenizer to construct the joint retraining framework. We strategically selected several representative combinations as baselines: (1) **Both w/o Retraining**, (2) **Tokenizer fine-tuning** with frozen RecLLM, (3) **RecLLM fine-tuning** with frozen tokenizer, (4) **Both fine-tuning**, (5) **Both full retraining** and (6) **RecLLM full retraining and tokenizer fine-tuning**. We additionally introduce (7) **LSAT** [24], a method that combines predictions from a fully retrained model and a model trained on current-period data. For its tokenizer, we employ the fine-tuning strategy. To demonstrate the robustness of our method,

Table 4: Overall performance of baselines and Reformer on TIGER. The best results are in bold and the second-best results are underlined. "LLM" and "Tok." is the abbreviation for the RecLLM and tokenizer, while "✕", "FT" and "RT" refer to "w/o Retraining", "Fine-tuning" and "Full retraining" respectively.

	Method		Period 1				Period 2				Period 3				Period 4			
Dataset	LLM	Tok.	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
Games	✕	✕	**0.0012**	0.0020	**0.0017**	**0.0018**	0.0009	0.0013	0.0011	0.0012	0.0003	0.0005	0.0003	0.0005	0.0001	0.0003	0.0001	0.0003
	✕	FT	**0.0012**	0.0020	**0.0017**	**0.0018**	0.0014	0.0023	0.0012	0.0015	0.0004	0.0006	0.0004	0.0004	0.0003	0.0003	0.0001	0.0002
	FT	✕	**0.0012**	0.0020	**0.0017**	**0.0018**	0.0062	0.0084	0.0080	0.0082	0.0037	0.0050	0.0036	0.0039	0.0098	0.0125	0.0087	0.0095
	FT	FT	**0.0012**	0.0020	**0.0017**	**0.0018**	0.0080	0.0108	**0.0107**	**0.0110**	0.0044	0.0127	0.0042	0.0076	0.0104	0.0167	0.0090	0.0113
	RT	RT	**0.0012**	0.0020	**0.0017**	**0.0018**	0.0065	0.0087	0.0090	0.0092	0.0053	0.0093	0.0057	0.0070	0.0110	0.0141	0.0097	0.0110
	RT	FT	**0.0012**	0.0020	**0.0017**	**0.0018**	0.0061	0.0085	0.0073	0.0077	0.0058	0.0086	0.0057	0.0065	0.0053	0.0105	0.0052	0.0072
	LSAT		**0.0012**	0.0020	**0.0017**	**0.0018**	**0.0086**	**0.0121**	0.0092	0.0100	0.0056	0.0105	0.0053	0.0071	0.0079	0.0152	0.0068	0.0096
	Reformer		**0.0012**	**0.0026**	0.0012	0.0017	0.0085	0.0118	0.0098	0.0105	**0.0076**	**0.0138**	**0.0060**	**0.0084**	**0.0131**	**0.0183**	**0.0128**	**0.0144**
Software	✕	✕	0.0173	0.0295	0.0133	0.0178	0.0086	0.0144	0.0066	0.0087	0.0067	0.0111	0.0051	0.0067	0.0040	0.0065	0.0036	0.0045
	✕	FT	0.0173	0.0295	0.0133	0.0178	0.0005	0.0014	0.0003	0.0007	0.0001	0.0002	0.0002	0.0002	0.0001	0.0006	0.0001	0.0003
	FT	✕	0.0173	0.0295	0.0133	0.0178	0.0396	0.0602	0.0400	0.0472	0.0412	0.0658	0.0381	0.0470	0.0352	0.0587	0.0307	0.039
	FT	FT	0.0173	0.0295	0.0133	0.0178	0.0434	0.0659	0.0427	0.0509	0.0427	0.0695	0.0370	0.0469	0.0406	0.0660	0.0383	0.0469
	RT	RT	0.0173	0.0295	0.0133	0.0178	0.0418	0.0610	0.0411	0.0477	0.0403	0.0656	0.0376	0.0466	0.0386	0.0606	0.0367	0.0439
	RT	FT	0.0173	0.0295	0.0133	0.0178	0.0406	0.0580	0.0357	0.0417	0.0405	0.0650	0.0377	0.0466	0.0386	0.0611	0.0362	0.0437
	LSAT		0.0173	0.0295	0.0133	0.0178	**0.0468**	**0.0677**	0.0423	0.0497	**0.0436**	0.0721	**0.0394**	0.0499	0.0365	0.0665	0.0328	0.0436
	Reformer		**0.0199**	**0.0426**	**0.0153**	**0.0178**	0.0467	0.0659	**0.0449**	**0.0513**	0.0431	**0.0751**	**0.0394**	**0.0516**	**0.0436**	**0.0700**	**0.0404**	**0.0492**
Steam	✕	✕	0.0181	**0.0328**	0.0413	0.0425	0.0105	0.0180	0.0220	0.0225	0.0077	0.0133	0.0159	0.0165	0.0062	0.0106	0.0137	0.0136
	✕	FT	0.0181	**0.0328**	0.0413	0.0425	0.0004	0.0010	0.0011	0.0012	0.0002	0.0005	0.0005	0.0006	0.0002	0.0004	0.0004	0.0004
	FT	✕	0.0181	**0.0328**	0.0413	0.0425	0.0243	0.0377	0.0466	0.0458	0.0205	0.0379	0.0336	0.0380	0.0198	0.0346	0.0378	0.0384
	FT	FT	0.0181	**0.0328**	0.0413	0.0425	0.0231	0.0357	**0.0470**	0.0451	0.0197	0.0365	0.0318	0.0361	0.0167	0.0312	0.0303	0.0329
	RT	RT	0.0181	**0.0328**	0.0413	0.0425	0.0222	0.0376	0.0436	0.0448	0.0191	0.0340	0.0344	0.0369	0.0182	0.0315	0.0350	0.0357
	RT	FT	0.0181	**0.0328**	0.0413	0.0425	0.0240	0.0382	0.0442	0.0442	0.0191	0.0353	0.0336	0.0369	0.0172	0.0306	0.0331	0.0346
	LSAT		0.0181	**0.0328**	0.0413	0.0425	0.0248	0.0395	0.0445	0.0452	0.0214	0.0387	0.0362	0.0400	0.0175	0.0326	0.0335	0.0356
	Reformer		**0.0197**	**0.0328**	**0.0458**	**0.0444**	**0.0252**	**0.0396**	0.0468	**0.0471**	**0.0220**	**0.0389**	**0.0389**	**0.0421**	**0.0228**	**0.0380**	**0.0421**	**0.0423**

we apply two codebook-based LLM4Rec methods. (1) **TIGER** [20] is a representative work in the codebook-based LLM4Rec paradigm useing RQ-VAE [11] to compress item semantic information into hierarchical token sequences. Built upon TIGER, (2) **LC-Rec** [39] introduces multiple recommendation-language alignment tasks and achieves performance optimization on a larger decoder-only model.

- **Evaluation metrics**. We use two metrics NDCG@K (N@K) and Recall@K (R@K) with K=5 and 10, following [20]. In particular, when an identifier is generated, we randomly select an item from the corresponding items as the recommendation result.
- **Implementation Details**. For training the RQ-VAE, we adopt a 3-layer codebook with 64 vectors of 32 dimensions in each layer. Following [30], we employ the AdamW optimizer with 1024 batch size and 0.001 learning rate for 20k training epochs and set μ to 0.25. Regarding the implementation of Reformer, the penalty coefficients α and $\beta \in \{0.1, 0.25, 0.5, None\}$ and $N_p \in \{8, 16, 32, 64\}$. Notably, the penalty constraint is applied during the entire training period.

5.2 Overall Performance (RQ1)

5.2.1 ***Performance on TIGER***. The comparison between our method and the baselines on the TIGER backbone implemented on T5-small is presented in Table 4, from which we can observe that:

- The performance of freezing RecLLM exhibits a widening gap to other baselines, emphasizing the necessity of RecLLM retraining due to dynamic item and user adaptation requirements.
- Fine-tuning both tokenizer and RecLLM outperforms RecLLM fine-tuning on the Games and Software datasets while performing worse on Steam, suggesting domain-dependent effectiveness correlated with feature shift strength. In domains with significant shifts (*e.g.,* e-commerce [31]), fine-tuning the tokenizer can enhance its adaptation to new items. Conversely, for relatively stable feature distributions, the frozen tokenizer maintains distinguishable capability for new items, while fine-tuning introduces identifier shifts that degrade performance.
- RecLLM Fine-tuning occasionally outperforms full retraining. This is because of the different modeling aspects of user interests. Full retraining captures users' long-term patterns, whereas fine-tuning focuses on short-term interests [24]. However, due to high computational cost, retraining RecLLM with the full retraining strategy is often impractical for real-world applications.
- LSAT demonstrates superior performance compared to RecLLM full retraining and tokenizer fine-tuning, primarily due to its incorporation of short-term user interest information. However, LSAT requires full retraining of RecLLM to capture long-term patterns, thereby limiting its practical applicability.
- Reformer generally outperforms all baselines across periods, with pronounced advantages in periods 3 and 4, demonstrating its effectiveness for long interaction sequences. This superiority stems from our strategy of (1) freezing identifiers that preserve the historical knowledge of RecLLM, enabling efficient retraining and (2) dynamic codebook expansion with frequency-based penalties that enable robust adaptation to new items.

5.2.2 ***Performance on LC-Rec.*** The comparison between our method and the baselines on LC-Rec is presented in Table 5. The following observations can be made:

- All baselines exhibit lower performance on LC-Rec than TIGER, likely due to Qwen-1.5B's a larger number of parameters with rich pre-trained knowledge that complicate adaptation to recommendation tasks with new vocabulary. Nevertheless, baseline performance trends remain consistent with TIGER.
- Consistent with observations in TIGER, Reformer still maintains superiority on LC-Rec. Notably, Both full retraining achieves comparable performance in period 4 of Games, indicating the

Table 5: Overall performance of baselines and Reformer instantiated on LC-Rec.

Dataset	Method		Period 1				Period 2				Period 3				Period 4			
	LLM	Tok.	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
Games	×	×	0.0014	0.0028	0.0014	0.0019	0.0009	0.0015	0.0011	0.0013	0.0002	0.0005	0.0003	0.0004	0.0001	0.0003	0.0002	0.0001
	×	FT	0.0014	0.0028	0.0014	0.0019	0.0004	0.0010	0.0003	0.0007	0.0002	0.0004	0.0002	0.0002	0.0001	0.0003	0.0001	0.0002
	FT	×	0.0014	0.0028	0.0014	0.0019	0.0030	0.0047	0.0036	0.0040	0.0036	0.0055	0.0037	0.0042	0.0044	0.0096	0.0043	0.0063
	FT	FT	0.0014	0.0028	0.0014	0.0019	0.0030	0.0052	0.0036	0.0043	0.0042	0.0087	0.0043	0.0060	0.0066	0.0120	0.0057	0.0078
	RT	RT	0.0014	0.0028	0.0014	0.0019	0.0017	0.0034	0.0020	0.0026	0.0028	0.0041	0.0035	0.0038	**0.0072**	0.0108	**0.0072**	0.0083
	RT	FT	0.0014	0.0028	0.0014	0.0019	0.0025	0.0069	0.0030	0.0049	0.0049	0.0079	0.0049	0.0059	0.0036	0.0079	0.0029	0.0046
	LSAT		0.0014	0.0028	0.0014	0.0019	0.0042	0.0078	0.0047	0.0057	0.0055	0.0090	0.0057	0.0071	0.0057	0.0092	0.0056	0.0067
	Reformer		**0.0026**	**0.0046**	**0.0026**	**0.0033**	**0.0061**	**0.0101**	**0.0064**	**0.0077**	**0.0075**	**0.0108**	**0.0068**	**0.0079**	**0.0072**	**0.0139**	0.0062	**0.0088**
Software	×	×	**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0054	0.0108	0.0045	0.0065	0.0031	0.0074	0.0027	0.0045	0.0012	0.0044	0.0010	0.0024
	×	FT	**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0020	0.0031	0.0013	0.0018	0.0010	0.0014	0.0011	0.0012	0.0003	0.0006	0.0004	0.0005
	FT	×	**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0168	0.0426	0.0135	0.0238	0.0286	0.0524	0.0263	0.0357	0.0262	0.0478	0.0231	0.0310
	FT	FT	**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0219	0.0413	0.0180	0.0253	0.0293	0.0555	0.0253	0.0355	0.0255	0.0488	0.0223	0.0310
	RT	RT	**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0250	0.0463	0.0210	0.0290	0.0249	0.0460	0.0216	0.0295	0.0242	0.0455	0.0222	0.0299
	RT	FT	**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0217	0.0400	0.0222	0.0274	0.0210	0.0437	0.0178	0.0268	0.0259	0.0496	0.0233	0.0320
	LSAT		**0.0151**	**0.0293**	**0.0118**	**0.0173**	0.0247	0.0483	**0.0249**	0.0298	0.0234	0.0495	0.0198	0.0316	0.0262	0.0502	0.0236	0.0318
	Reformer		0.0136	0.0279	0.0101	0.0158	**0.0286**	**0.0538**	0.0221	**0.0320**	**0.0304**	**0.0601**	**0.0272**	**0.0389**	**0.0293**	**0.0515**	**0.0252**	**0.0335**

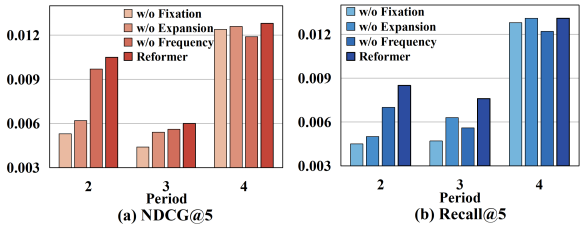


Figure 4: Ablation study of Reformer with TIGER on Games.

Qwen-1.5B's larger parameter size enables effective interaction modeling with sufficient interaction data.

5.3 In-depth Analysis

5.3.1 ***Ablation Study (RQ2).*** We verify the effectiveness of each component in Reformer by individually removing the dynamic codebook expansion strategy (w/o Expansion), the identifier freezing strategy (w/o Fixation), and the frequency penalty term (w/o Frequency). The results in Figure 4 show that: **(1)** Reformer generally achieves optimal performance, confirming each component's necessity. **(2)** The Removal of the identifier freezing strategy causes significant performance degradation due to identifier shifts, demonstrating the importance of identifier stability.

5.3.2 ***Item Group Analysis (RQ3).*** To investigate how Reformer improves performance, we evaluate Reformer's performance on warm and cold items compared to two competitive baselines (Figure 5), where items present in the tokenizer's training data are defined as warm items, while others are considered cold items.

From Figure 5, we can find that **(1)** RecLLM fine-tuning outperforms both fine-tuning on warm items in period 2, but degrades in later periods as accumulating new items move beyond its representation space *cf.* Section 3.2. **(2)** Both fine-tuning method outperforms RecLLM fine-tuning in period 2 but yields inferior performance in subsequent periods, a possible reason is the identifier shifts issue. **(3)** The improvements of Reformer on warm items are probably due to the identifier freezing strategy, which effectively preserves the RecLLM's understanding of warm items. **(4)** For cold items, Reformer shows improvements in period 2/4, but a comparable performance is observed in period 3. We suggest that the cold item distribution in period 3 may resemble that of period 0, allowing the tokenizer to perform well without retraining.

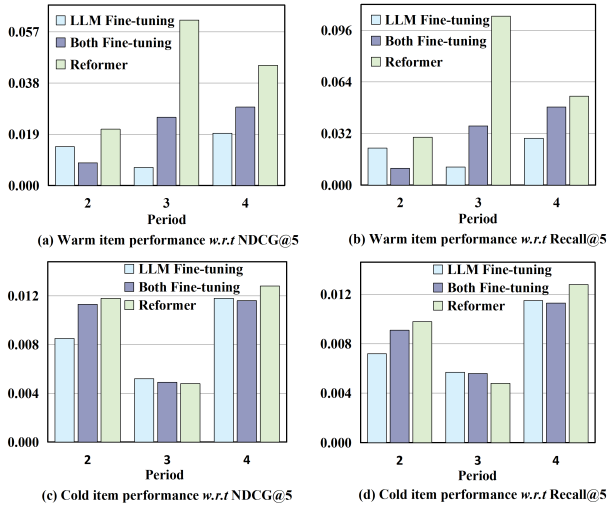


Figure 5: Performance of Reformer, both fine-tuning, and RecLLM fine-tuning on warm and cold items of Games.

5.3.3 ***Codebook Expansion Size Analysis (RQ3).*** To investigate the impact of codebook expansion size, we conduct experiments with codebook expansion size $N_p \in \{8, 16, 32, 64\}$ and other hyperparameters fixed. The results are shown in Figures 6 (a), (b), and (c). The key observations are as follows: **(1)** In periods 2 and 4, an appropriate codebook size (*i.e.,* N_p = 8 or 16) leads to better recommendation performance, confirming dynamic expansion's benefits for new item adaptation. However, Oversized codebooks degrade the performance due to the sparsity of the new items' distribution, which may result in overfitting, consistent with [30]. **(2)** Performance fluctuates with N_p in period 3, likely because the item distribution in this period resembles that of period 0 (backed up Section 5.3.2), reducing the necessity of codebook expansion.

5.3.4 ***Penalty Term Analysis (RQ3).*** We analyze the penalty strength effects by fixing other hyperparameters and set the penalty for new codebooks to 0 while varying the penalty for old codebooks $\alpha \in \{0.1, 0.25, 0.5, None\}$. The corresponding results in Figures 6 (d), (c), and (e) show that: (1) Appropriate penalty strength outperforms no penalty, demonstrating that the enhancement of diversity achieves a better understanding of new items for RecLLM. (2) Excessive penalties degrade performance as the unpenalized new

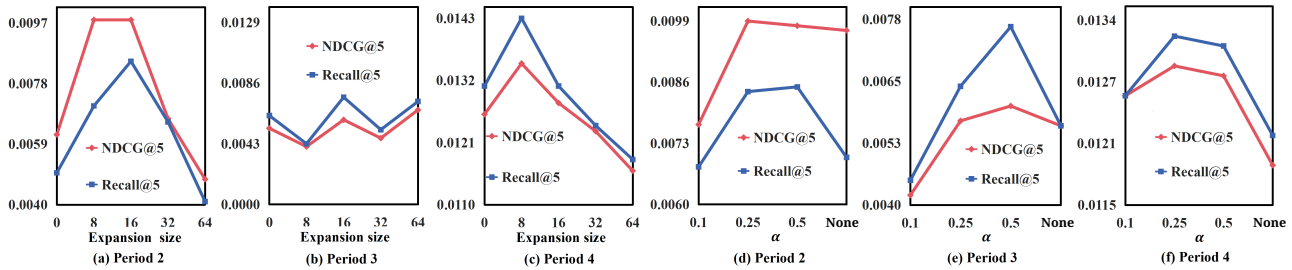


Figure 6: Performance of Reformer with different strengths of the penalty and different sizes of expanded codes.

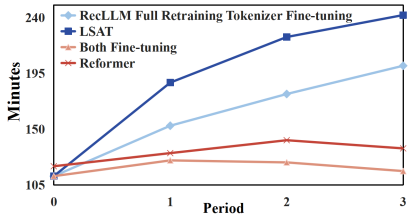


Figure 7: Overall Time Consumption of Reformer and Baselines with TIGER on Games.

codebook attracts excessive items assignment, thereby enhancing the bias of identifiers and leading to a decline in the tokenizer's representation capability.

5.3.5 ***Time Efficiency Analysis (RQ4).*** To evaluate Reformer's efficiency, we compare its total training and inference time per period against representative baselines for both the tokenizer and RecLLM. The results are presented in Figure 7, the key findings are as follows: **(1)** Although Reformer introduces additional overhead due to its frequency penalty, its runtime remains acceptable versus the fastest fine-tuning baseline. **(2)** LSAT's joint training of short- and long-term models leads to notably higher time complexity. Along with earlier performance comparisons, our method outperforms LSAT in both efficiency and effectiveness.

6 Related Work

• **LLM-based Recommendation.** Existing studies [6, 13, 25, 26, 36] can be broadly categorized into two types: 1) LLM-enhanced traditional recommenders [4, 10, 17, 19, 22, 28, 32, 33, 38, 40], such as LLMs for feature construction [21, 34] and information extraction [25]; (2) LLMs as recommenders [2, 13, 15, 20, 30] which typically uses a tokenizer to map items to identifiers and convert user interactions into token sequences, which are then input into RecLLMs for recommendation. Existing tokenizers can be categorized as follows: (1) unlearnable tokenizer [2, 3, 7, 8] assigning an ID or a text sequence to each item. However, fixed identifiers limit the ability to encode rich item representations. Thus, (2) learnable tokenizers [14, 20, 30, 39] are proposed to adopt a neural model to convert item information into the identifier. Existing learnable tokenization methods can be mainly divided into: *ID identifier* [14] assigns each item a unique ID. However, due to the lack of semantic understanding, these methods often fail to achieve optimal performance. In contrast, *textual identifiers* [27] construct identifiers from item textual information. However, textual identifiers may fail to capture semantics from coarse-grained to fine-grained levels, leading to misalignment with the autoregressive generation process of RecLLMs. Recently, *codebook-based identifiers* [20, 30, 39] have been proposed to use a small number of codes to represent extensive items, achieving effective information compression through hierarchical encoding and leading to high recommendation performance [39]. As such, we primarily develop Reformer on codebook-based tokenizers and defer the adaptation of Reformer to other learnable tokenizers for future work.

• **Incremental Learning**. Incremental learning is essential for recommender systems to adapt to evolving items and user interactions [35, 37]. While well-studied for traditional recommenders [5, 29, 35, 37], incremental learning for LLM-based recommenders presents new challenges. For instance, LSAT [24] employs LoRA modules to integrate long- and short-term user interests for performance enhancement, while RecICL-TALLRec [1] adopts in-context learning to for parameter-free incremental learning in LLM4Rec. However, these approaches are based on unlearnable tokenizers, for which previous retraining work can be easily adapted. In contrast, this work investigates more challenging retraining on learnable tokenizers, identifies two crucial challenges, and proposes an effective retraining framework.

7 Conclusion and Future work

In this work, we focused on a critical yet under-explored task: retraining RecLLMs with learnable tokenizers. Through comprehensive empirical and theoretical analyses, we identified two key issues of jointly retraining RecLLMs and tokenizers: identifier collision and identifier shifts. To tackle these issues, we proposed Reformer, which integrates a dynamic codebook with a diversity constraint to prevent identifier collision and an identifier freezing strategy to maintain identifier invariance. Extensive experiments on two representative RecLLMs and three real-world datasets validate the effectiveness of Reformer, paving the way for the real-world deployment of RecLLMs.

As an early study on retraining RecLLMs with learnable tokenizers, our work opens several promising directions for future research. First, data-efficient fine-tuning strategies can be explored to selectively update RecLLMs using only representative samples at each period, improving efficiency while maintaining recommendation accuracy. Second, given the performance superiority of codebook-based tokenizers [39], our experiments primarily focus on TIGER and LC-Rec; future work could adapt Reformer to other learnable tokenizers, such as IDGenRec [27]. Third, as user interests and item distributions evolve, integrating LLMs' knowledge editing techniques into the incremental learning process might refine outdated user preferences learned from historical data, reducing the new data requirement for superior efficiency.

Table 6: Overall performance of baselines and Reformer on TIGER of Games dataset. The best results are in bold and the second-best results are underlined.

	Method		Period 1				Period 2				Period 3				Period 4			
Dataset	LLM	Tok.	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
Games	FT	×	**0.0052**	0.0075	**0.0050**	**0.0057**	0.0111	0.0210	0.0105	0.0142	0.0045	0.0082	0.0044	0.0058	0.0132	0.0191	0.0145	0.0162
	FT	FT	**0.0052**	0.0075	**0.0050**	**0.0057**	0.0122	0.0173	**0.0116**	0.0131	0.0055	0.0086	0.0061	0.0071	0.0137	0.0184	0.0148	0.0160
	RT	RT	**0.0052**	0.0075	**0.0050**	**0.0057**	0.0096	0.0135	0.0097	0.0107	0.0063	0.0070	0.0053	0.0058	0.0096	0.0139	0.0101	0.0113
	RT	FT	**0.0052**	0.0075	**0.0050**	**0.0057**	0.0119	0.0190	0.0112	0.0134	0.0058	0.0081	0.0050	0.0057	0.0091	0.0141	0.0094	0.0109
	LSAT		**0.0052**	0.0075	**0.0050**	**0.0057**	0.0120	0.0213	0.0107	0.0144	0.0054	0.0112	0.0052	0.0074	0.0142	0.0200	0.0156	0.0171
	Reformer		0.0038	0.0057	0.0040	0.0046	**0.0131**	**0.0216**	0.0115	0.0146	**0.0065**	**0.0140**	**0.0059**	**0.0089**	**0.0154**	**0.0210**	**0.0162**	**0.0178**

A Addiotnal Experiments

We additionally provide results of the Games dataset divided by equal interactions on TIGER presented in Table 6. The results demonstrate that the performance of both Reformer and baselines exhibit trends almost consistent with Table 4.

B Proof of Theorem 3.1

Proof. Since the codebook is optimized to achieve uniform quantization assignments for data stream I_1, we assume that the d-dimensional space is partitioned into N equal-probability regions $V_k{}_{k=1}^{N}$. Each region V_k corresponds to a generator point $\mathbf{u}_k$ on the unit sphere S^{d-1}, defined by:

$$V_k = \{\mathbf{x} \in \mathbb{R}^d \mid \arccos\left(\frac{\mathbf{x}\cdot\mathbf{u}_k}{|\mathbf{x}|}\right) \le \theta\} \tag{14}$$

where θ is determined by equal angular division. Without loss of generality, we assume that data stream I_2 has embeddings distributed as $\mathcal{N}(\mathbf{u}, \sigma^2\mathbf{I})$ with $\mathbf{u} = a\mathbf{u}_k$. To derive a lower bound for the probability integral over region V_k:

$$P_k = \int_{V_k} \frac{1}{(2\pi\sigma^2)^{d/2}} e^{-|\mathbf{x}-\mathbf{u}|^2/(2\sigma^2)} d\mathbf{x}, \tag{15}$$

we decompose the probability as:

$$P_k = 1 - \Pr\left(\arccos\left(\frac{\mathbf{x}\cdot\mathbf{u}_k}{|\mathbf{x}|}\right) > \theta\right). \tag{16}$$

we transform it into:

$$P_k \ge 1 - \Pr\left(\frac{\mathbf{x}\cdot\mathbf{u}_k}{|\mathbf{x}|} < \theta'\right), \tag{17}$$

where $\theta' = \cos(\theta)$.

Step 1: Decomposition of x. The vector $\mathbf{x}$ can be decomposed into components along the direction of $\mathbf{u}$ and perpendicular to it:

$$\mathbf{x} = x_\parallel + x_\perp. \tag{18}$$

Here, the parallel component follows:

$$x_\parallel \sim \mathcal{N}(a, \sigma^2), \tag{19}$$

while the perpendicular component follows:

$$|x_\perp|^2 \sim \sigma^2 \chi^2_{d-1}. \tag{20}$$

Step 2: Reformulating the Probability Expression. We rewrite the probability condition as:

$$\Pr\left(x_\parallel \le \theta' \sqrt{x_\parallel^2 + |x_\perp|^2}\right). \tag{21}$$

Squaring both sides leads to:

$$x_\parallel^2 \le \frac{\theta'^2}{1-\theta'^2} |x_\perp|^2. \tag{22}$$

This simplifies to:

$$\frac{x_\parallel}{|x_\perp|} \le \frac{\theta'}{\sqrt{1-\theta'^2}}. \tag{23}$$

Step 3: Applying Chernoff Bound. The probability we need to control is

$$\Pr\left(x_\parallel \le \cot\theta |x_\perp|\right). \tag{24}$$

For any fixed $r \ge 0$, we have

$$\Pr\left(x_\parallel \le \cot\theta r \mid |x_\perp| = r\right) = \Phi\left(\frac{\cot\theta r - a}{\sigma}\right), \tag{25}$$

where $\Phi(\cdot)$ is the CDF of the standard normal distribution. Using a standard tail bound,

$$\Phi\left(\frac{\cot\theta r - a}{\sigma}\right) \le \exp\left(-\frac{(a-\cot\theta r)^2}{2\sigma^2}\right). \tag{26}$$

Next, we note that $|x_\perp|$ is a (scaled) chi random variable that concentrates around its mean. A standard concentration bound implies that for any $\delta > 0$,

$$\Pr\left(|x_\perp| \ge \sigma(\sqrt{d-1} + \delta)\right) \le \exp\left(-\frac{\delta^2}{2}\right). \tag{27}$$

Setting

$$\delta = \frac{a}{\sigma} - \cot\theta\sqrt{d-1}, \tag{28}$$

which is positive provided that $\frac{a}{\sigma} > \cot\theta\sqrt{d-1}$, we conclude that with high probability,

$$|x_\perp| \le \frac{a}{\cot\theta}. \tag{29}$$

Thus, on this high-probability event,

$$\Pr\left(x_\parallel \le \cot\theta |x_\perp|\right) \le \exp\left(-\frac{(a-\cot\theta|x_\perp|)^2}{2\sigma^2}\right). \tag{30}$$

Applying a union bound, we obtain

$$\Pr\left(\frac{x_\parallel}{|x_\perp|} \le \cot\theta\right) \le \exp\left(-\frac{1}{2}\left(\frac{a}{\sigma} - \cot\theta\sqrt{d-1}\right)^2\right). \tag{31}$$

Finally, substituting into our bound for P_k,

$$P_k \ge 1 - \exp\left(-\frac{1}{2}\left(\frac{a}{\sigma} - \cot\theta\sqrt{d-1}\right)^2\right). \tag{32}$$

Therefore, the probability of two items appearing in the region V_k (i.e., the collision probability) is

$$col_rate \ge 1 - (1-P_{k_min})^{|I_2|} - |I_2| P_{k_min} (1-P_{k_min})^{|I_2|-1}, \tag{33}$$

$$P_{k_min} = 1 - \exp\left(-\frac{1}{2}\left(\frac{a}{\sigma} - \cot\theta\sqrt{d-1}\right)^2\right). \tag{34}$$

This completes the proof. □

GenAI Usage Disclosure

The authors used ChatGPT to assist with language refinement during the writing process of this work. After using this service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] Keqin Bao, Ming Yan, Yang Zhang, Jizhi Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2024. Real-Time Personalization for LLM-based Recommendation with Customized In-Context Learning. *arXiv preprint arXiv:2410.23136* (2024).

[2] Keqin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yanchen Luo, Chong Chen, Fuli Feng, and Qi Tian. 2025. A bi-step grounding paradigm for large language models in recommendation systems. *ACM Transactions on Recommender Systems* 3, 4 (2025), 1–27.

[3] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*. ACM.

[4] Lei Chen, Chen Gao, Xiaoyi Du, Hengliang Luo, Depeng Jin, Yong Li, and Meng Wang. 2024. Enhancing ID-based Recommendation with Large Language Models. (2024).

[5] Ernesto Diaz-Aviles, Lucas Drumond, Lars Schmidt-Thieme, and Wolfgang Nejdl. 2012. Real-time top-n recommendation in social streams. In *Proceedings of the sixth ACM conference on Recommender systems*. 59–66.

[6] Junchen Fu, Xuri Ge, Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, Jie Wang, and Joemon M Jose. 2024. IISAN: Efficiently adapting multimodal representation for sequential recommendation with decoupled PEFT. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 687–697.

[7] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*. 299–315.

[8] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to index item ids for recommendation foundation models. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 195–204.

[9] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[10] Sein Kim, Hongseok Kang, Seungyoon Choi, Donghyun Kim, Minchul Yang, and Chanyoung Park. 2024. Large language models meet collaborative filtering: An efficient all-round llm-based recommender system. In *KDD*. 1395–1406.

[11] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11523–11532.

[12] Hyunsung Lee, Sungwook Yoo, Dongjun Lee, and Jaekwang Kim. 2023. How Important is Periodic Model update in Recommender System?. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2661–2668.

[13] Peibo Li, Maarten de Rijke, Hao Xue, Shuang Ao, Yang Song, and Flora D Salim. 2024. Large language models for next point-of-interest recommendation. In *SIGIR*. 1463–1472.

[14] Xinhang Li, Chong Chen, Xiangyu Zhao, Yong Zhang, and Chunxiao Xing. 2023. E4SRec: An Elegant Effective Efficient Extensible Solution of Large Language Models for Sequential Recommendation. *arXiv:2312.02443*.

[15] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2024. Llara: Large language-recommendation assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1785–1795.

[16] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2023. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems* (2023).

[17] Xinyu Lin, Wenjie Wang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Bridging items and language: A transition paradigm for large language model-based recommendation. In *KDD*. ACM, 1816–1826.

[18] Xinyu Lin, Wenjie Wang, Yongqi Li, Shuo Yang, Fuli Feng, Yinwei Wei, and Tat-Seng Chua. 2024. Data-efficient Fine-tuning for LLM-based Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 365–374.

[19] Xinyu Lin, Wenjie Wang, Jujia Zhao, Yongqi Li, Fuli Feng, and Tat-Seng Chua. 2024. Temporally and distributionally robust optimization for cold-start recommendation. In *AAAI*, Vol. 38. 8750–8758.

[20] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, et al. 2023. Recommender Systems with Generative Retrieval. In *NeurIPS*. Curran Associates, Inc.

[21] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Representation learning with large language models for recommendation. In *WWW*. ACM.

[22] Yankun Ren, Zhongde Chen, Xinxing Yang, Longfei Li, Cong Jiang, Lei Cheng, Bo Zhang, Linjian Mo, and Jun Zhou. 2024. Enhancing sequential recommenders with augmented knowledge from aligned large language models. In *SIGIR*. 345–354.

[23] Steffen Rendle and Lars Schmidt-Thieme. 2008. Online-updating regularized kernel matrix factorization models for large-scale recommender systems. In *Proceedings of the 2008 ACM conference on Recommender systems*. 251–258.

[24] Tianhao Shi, Yang Zhang, Zhijian Xu, Chong Chen, Fuli Feng, Xiangnan He, and Qi Tian. 2024. Preliminary study on incremental learning for large language model-based recommender systems. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4051–4055.

[25] Wentao Shi, Xiangnan He, Yang Zhang, Chongming Gao, Xinyue Li, Jizhi Zhang, Qifan Wang, and Fuli Feng. 2024. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.

[26] Zhu Sun, Hongyang Liu, Xinghua Qu, Kaidong Feng, Yan Wang, and Yew Soon Ong. 2024. Large language models for intent-driven session recommendations. In *SIGIR*. 324–334.

[27] Juntao Tan, Shuyuan Xu, Wenyue Hua, Yingqiang Ge, Zelong Li, and Yongfeng Zhang. 2024. Idgenrec: Llm-recsys alignment with textual id learning. In *SIGIR*. 355–364.

[28] Jie Wang, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M Jose. 2024. Reinforcement learning-based recommender systems with large language models for state reward and action modeling. In *SIGIR*. 375–385.

[29] Qinyong Wang, Hongzhi Yin, Zhiting Hu, Defu Lian, Hao Wang, and Zi Huang. 2018. Neural memory streaming recommender networks with adversarial training. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2467–2475.

[30] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable item tokenization for generative recommendation. In *CIKM*. 2400–2409.

[31] Wenjie Wang, Xinyu Lin, Liuhui Wang, Fuli Feng, Yunshan Ma, and Tat-Seng Chua. 2023. Causal disentangled recommendation against user preference shifts. *ACM Transactions on Information Systems* 42, 1 (2023), 1–27.

[32] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. EAGER: Two-Stream Generative Recommender with Behavior-Semantic Collaboration. In *KDD*. ACM, 3245–3254.

[33] Junda Wu, Cheng-Chun Chang, Tong Yu, Zhankui He, Jianing Wang, Yupeng Hou, and Julian McAuley. 2024. Coral: collaborative retrieval-augmented large language models improve long-tail recommendation. In *KDD*. ACM, 3391–3401.

[34] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *RecSys*. 12–22.

[35] Ruobing Xie, Yalong Wang, Rui Wang, Yuanfu Lu, Yuanhang Zou, Feng Xia, and Leyu Lin. 2022. Long short-term temporal meta-learning in online recommendation. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1168–1176.

[36] Peiyan Zhang, Yuchen Yan, Xi Zhang, Liying Kang, Chaozhuo Li, Feiran Huang, Senzhang Wang, and Sunghun Kim. 2024. Gpt4rec: Graph prompt tuning for streaming recommendation. In *SIGIR*. 1774–1784.

[37] Yang Zhang, Fuli Feng, Chenxu Wang, Xiangnan He, Meng Wang, Yan Li, and Yongdong Zhang. 2020. How to retrain recommender system? A sequential meta-learning method. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1479–1488.

[38] Yuyue Zhao, Jiancan Wu, Xiang Wang, Wei Tang, Dingxian Wang, and Maarten de Rijke. 2024. Let me do it for you: Towards llm empowered recommendation via tool learning. In *SIGIR*. 1796–1806.

[39] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.

[40] Zhi Zheng, Wenshuo Chao, Zhaopeng Qiu, Hengshu Zhu, and Hui Xiong. 2024. Harnessing large language models for text-rich sequential recommendation. In *WWW*. 3207–3216.